

Proyecto 1 — Chatbot anfitrión con Model Context Protocol

CC3067 Redes · Universidad del Valle de Guatemala

Andrés Mazariegos

17 de septiembre de 2026

Table of Contents

Proyecto 1 — Chatbot anfitrión con Model Context Protocol.....	2
1. Qué se construyó.....	2
2. Especificación del servidor propio (requisito 8)	3
2.1 Identidad y ciclo de vida	3
2.2 Herramientas y parámetros	3
2.3 Endpoints	4
2.4 De dónde salen los datos.....	5
2.5 La distinción que gobierna todo el diseño.....	6
3. Análisis por capas (requisito 9).....	6
3.1 Por qué hay dos capturas y no una	6
3.2 Capa de enlace — Ethernet II	7
3.3 Capa de red — IPv4	8
3.4 Capa de transporte — TCP.....	8
3.5 Capa de aplicación — HTTP, JSON-RPC, y lo que TLS oculta	9
3.6 Clasificación de los mensajes (requisito 7).....	9
3.7 Cómo se anidan las capas	10
4. La interfaz y su justificación de HCI (punto extra)	11
4.1 Feedback inmediato · visibilidad del estado del sistema	11
4.2 Psicología del color · y por qué el color nunca va solo	12
4.3 Agrupación · principios de Gestalt.....	12
4.4 Prevención de errores.....	13
4.5 Reconocimiento antes que recuerdo	13
4.6 Dos errores que solo aparecieron al mirar	13
5. Cumplimiento de los requisitos.....	14

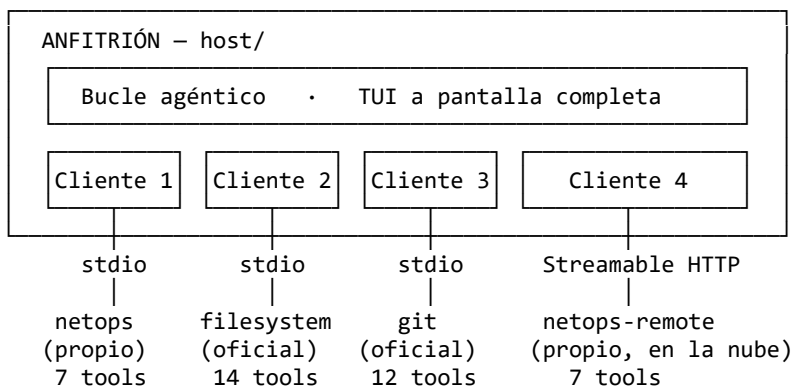
5.1 La prueba de que local y remoto son el mismo servidor	15
6. Conclusiones y comentarios (requisito 10)	15
6.1 Lo que el proyecto enseñó sobre el protocolo.....	15
6.2 Lo que salió mal, y qué se aprendió.....	16
6.3 Limitaciones conocidas.....	16
6.4 Comentario final	17
Referencias.....	17

Proyecto 1 — Chatbot anfitrión con Model Context Protocol

Curso: CC3067 Redes · Universidad del Valle de Guatemala **Autor:** Andrés Mazariegos **Repositorio:** <https://github.com/andresm220/Proyecto1Redes> **Servidor desplegado:** <https://netops-mcp-261683462697.us-central1.run.app>

1. Qué se construyó

Un chatbot de terminal que actúa como **anfitrión (host) MCP**: mantiene una conversación con un modelo de lenguaje, y cuando el modelo necesita un dato que no tiene, lo pide a alguno de los cuatro servidores MCP conectados.



40 herramientas agregadas, dos transportes, cero SDK de MCP. Todo el formato e intercambio de mensajes JSON-RPC 2.0 —framing, handshake, requests, responses, notifications, errores— está escrito a mano. Un test del repositorio verifica que `requirements.txt` no contenga ninguna librería de MCP.

Métrica	Valor
Código de producción	5,510 líneas
Código de pruebas	4,256 líneas
Pruebas automatizadas	364, todas en verde
Verificación de conformidad	19/19 en tres entornos distintos
Commits	34, entre el 19 de agosto y el 17 de septiembre

2. Especificación del servidor propio (requisito 8)

netops es el back office de soporte técnico de un proveedor de internet guatemalteco. Se eligió esa industria porque es real, es temáticamente afín al curso, y porque su lógica de decisión es lo bastante rica como para que el modelo tenga algo interesante que hacer con ella.

2.1 Identidad y ciclo de vida

Aspecto	Valor
Nombre	netops
Versión	1.0.0
Versión de protocolo	2025-11-25
Capacidades	{"tools": {"listChanged": false}}
Métodos servidos	initialize, notifications/initialized, tools/list, tools/call, ping
Cualquier otro método	-32601 Method not found

El handshake exige sus **dos mitades y en orden**. Rastrear solo la notificación permitiría que un cliente se saltara initialize por completo y aun así fuera tratado como inicializado, así que se llevan dos banderas separadas: initialize_received e initialized.

2.2 Herramientas y parámetros

Un asterisco marca los obligatorios; entre llaves van los valores admitidos.

Herramienta	Parámetros	Devuelve
lookup_account	account_id:string, phone:string	plan, estado, dirección de servicio
check_service_status	account_id*:string	estado del enlace y métricas
list_outages	region:{guatemala\ quetzalteango\ peten\ escuintla}, active_only:boolean	incidencias masivas con ETA

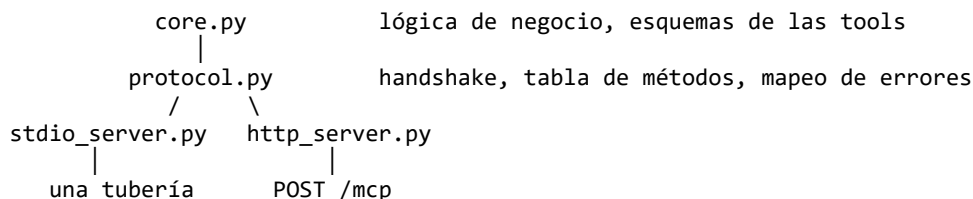
Herramienta	Parámetros	Devuelve
run_diagnostic	account_id*:string, test_type*:{ping\ speed\ line}	mediciones y causa probable
open_ticket	account_id*, category*:{connectivity\ speed\ billing\ equipment\ installation}, description*, priority:{low\ normal\ high\ critical}	id del ticket
get_ticket	ticket_id*:string	estado e historial
schedule_visit	ticket_id*, date*:YYYY-MM-DD, time_window*:{08:00-12:00\ 12:00-16:00\ 16:00-20:00}	confirmación de la visita

lookup_account es el único caso que JSON Schema no expresa por sí solo: acepta account_id o phone, exactamente uno. Eso requeriría oneOf, que el validador propio no implementa, así que la regla vive en una función explícita, require_one_of, y falla con -32602.

La especificación completa, con los esquemas íntegros y ejemplos de request/response capturados de un servidor en ejecución, está en `servers/netops/SPEC.md`.

2.3 Endpoints

El mismo servidor se alcanza por dos caminos. No son dos implementaciones: son dos adaptadores sobre un mismo motor.



stdio. El anfitrión lanza el servidor como subproceso y habla por las entradas y salidas estándar. Framing **NDJSON**: un mensaje JSON por línea, terminado en `\n`, UTF-8, sin saltos de línea embebidos. No es el framing con Content-Length que usa LSP. `stdout` lleva protocolo y nada más; todo log va a `stderr`, de modo que un `print` perdido no puede corromper el flujo.

Streamable HTTP. Un solo endpoint, `POST /mcp`, con un mensaje JSON-RPC por petición.

Endpoint	Método	Para qué
/mcp	POST	todo el protocolo

Endpoint	Método	Para qué
/mcp	DELETE	terminar la sesión (opcional en la especificación, implementado)
/health	GET	sonda de vida de la plataforma

El código de estado dice **qué tipo de mensaje** recibió el servidor:

Recibió	Responde
un request	200 con Content-Type: application/json y la respuesta
una notification	202 Accepted sin cuerpo, porque no debe ninguna
una response	202 Accepted, igual

Cabeceras: `Mcp-Session-Id` se emite en el `initialize` y se exige después; `MCP-Protocol-Version` acompaña todo lo posterior al handshake.

Las sesiones son lo único que HTTP necesita y stdio no. Una tubería es la sesión: empieza con el proceso y termina con él. Las peticiones HTTP llegan independientes y tienen que decir a qué conversación pertenecen. Un id desconocido responde 404, que es la señal que la especificación define para que el cliente abra una sesión nueva en vez de reintentar.

2.4 De dónde salen los datos

Tres fuentes, y la separación es deliberada.

Semilla versionada. `data/seed/accounts.json` y `outages.json`: cinco suscriptores y tres incidencias, escritos a mano, en git, que el servidor nunca reescribe. Las cinco cuentas cubren un caso de prueba cada una — una normal, una suspendida por mora, una dentro de una falla masiva, una de plan alto, y una pendiente de instalación.

Estado mutable. `data/state.json`: solo tickets, visitas y el contador. Se escribe de forma atómica —archivo temporal, `fsync`, `os.replace`— así que un corte a media escritura deja intacto el archivo viejo en vez de uno truncado. Si llega corrupto, el servidor arranca limpio en vez de morir.

Métricas derivadas. La latencia, la pérdida de paquetes, el SNR y la velocidad **no están almacenadas en ningún archivo**. Se calculan al vuelo desde un hash SHA-256 del identificador de cuenta:

```
def stable_int(*parts, low, high):
    digest = hashlib.sha256("".join(parts).encode("utf-8")).digest()
    return low + (int.from_bytes(digest[:8], "big") % (high - low + 1))
```

Cada métrica usa una etiqueta distinta como sal, de modo que salen valores independientes de la misma cuenta. Lo que está fijo en el código son los **rangos**, no los valores: latencia entre 6 y 45 ms, SNR entre 24 y 38 dB, velocidad de bajada entre 82% y 99% del plan contratado.

La razón de no usar `random` es la reproducibilidad. Con `random`, este documento diría 31 ms y la demo mostraría otra cosa. Con el hash, GT-10233 reporta `lat=31ms loss=0.6% snr=34.2dB` hoy, mañana, y en el servidor de la nube — que es, de paso, una manera de comprobar que local y remoto ejecutan el mismo código.

2.5 La distinción que gobierna todo el diseño

Este es el punto conceptual central del proyecto y conviene enunciarlo solo.

Un **error de protocolo** significa que el intercambio nunca estuvo bien formado: JSON inválido, método inexistente, argumentos que no satisfacen el esquema publicado. Va en el campo `error` de JSON-RPC, con código negativo.

Un **error de dominio** significa que la llamada estuvo perfecta y la operación no pudo tener éxito: la cuenta no existe, el ticket no existe, la fecha ya pasó. Va como `result exitoso` que carga `isError: true`.

```
// protocolo: el esquema exige account_id y no vino
{"jsonrpc": "2.0", "id": 3, "error": {"code": -32602,
  "message": "account_id: is required", "data": {"field": "account_id"}}}

// dominio: la llamada estuvo bien, la cuenta no existe
{"jsonrpc": "2.0", "id": 4, "result": {
  "content": [{"type": "text", "text": "\error\":"account_not_found", ...}],
  "isError": true}}
```

No es una sutileza académica: **es lo que permite que el modelo se recupere solo**. Un error de dominio vuelve al modelo como `tool_result` con `is_error`, lo lee, y corrige. Un error de protocolo significa que el host construyó mal la llamada, y eso es un bug nuestro, no algo que el modelo deba intentar rodear.

Los cinco códigos reservados están implementados: -32700 parse error, -32600 invalid request, -32601 method not found, -32602 invalid params, -32603 internal error.

3. Análisis por capas (requisito 9)

3.1 Por qué hay dos capturas y no una

Se tomaron dos capturas con `tshark`, y la razón de que sean dos es en sí misma un resultado que vale la pena reportar.

Cloud Run sirve únicamente HTTPS. Google termina el TLS y *.run.app no es direccionable por HTTP plano. Una captura contra el despliegue real muestra el handshake TLS y después Application Data cifrado: se ve que hubo una conversación y cuántos bytes costó, pero no cuál mensaje era tools/call — que es exactamente lo que el requisito 7 pide señalar.

Loopback entrega el texto en claro pero no tiene capa de enlace. Correr el mismo servidor en HTTP plano localmente y capturar en el adaptador de loopback de Npcap recupera todos los mensajes, pero las tramas no llevan cabecera Ethernet alguna:

```
Encapsulation type: NULL/Loopback (15)
[Protocols in frame: null:ip:tcp]
```

No hay direcciones MAC que reportar porque nunca se construyó una trama.

La tercera opción se descartó a conciencia. Texto plano sobre un segmento Ethernet real habría exigido que el servidor fuera alcanzable desde otro host —una segunda máquina, o el invitado WSL alcanzando de vuelta a Windows— y esto último requiere abrir un puerto entrante en el firewall. Debilitar el firewall de la máquina por comodidad de un ejercicio no es un intercambio que valga la pena.

Cada captura responde lo que puede responder con honestidad. La primera lleva la capa de aplicación; la segunda, todo lo que está debajo.

3.2 Capa de enlace — Ethernet II

De captures/mcp_cloudrun_tls.pcapng, 248 paquetes sobre Wi-Fi:

Trama	MAC origen	MAC destino	Sentido
280	3c:21:9c:ab:a2:61	08:95:2a:e2:fb:cb	salida
281	08:95:2a:e2:fb:cb	3c:21:9c:ab:a2:61	entrada

Ninguna de las dos MAC es la del servidor en Google, y ese es el punto. Una dirección MAC solo tiene significado dentro de un dominio de difusión.

3c:21:9c:ab:a2:61 es el adaptador Wi-Fi de esta máquina y 08:95:2a:e2:fb:cb es la puerta de enlace predeterminada: el primer salto. El servidor está a nueve saltos y su MAC nunca viajó por aquí ni podría hacerlo. Cada router del camino reescribe ambas MAC dejando las direcciones IP intactas, y esa diferencia es la razón entera de que las dos capas estén separadas.

Las tramas más grandes de la captura miden 1,466 bytes y cuadran al byte:

```
14  cabecera Ethernet II
20  cabecera IPv4
20  cabecera TCP
1412 carga TCP
-----
1466 bytes en el cable
```

Cómodamente dentro de la MTU de 1,500 de la interfaz, que es por lo que nada tuvo que fragmentarse.

3.3 Capa de red — IPv4

Campo	Salida	Entrada
Origen	192.168.0.29	34.143.73.2
Destino	34.143.73.2	192.168.0.29
TTL	128	119
Don't Fragment	activo	activo

El TTL insinúa la longitud del camino. Nuestros paquetes salen con 128, el valor por omisión de Windows. Las respuestas llegan con 119, y como de los valores iniciales habituales (64, 128, 255) solo 128 puede decrementarse hasta 119, **las respuestas atravesaron nueve routers**. Es una inferencia a partir de un valor inicial convencional, no algo que el paquete declare.

Nada se fragmentó: DF está activo en ambos sentidos, y TCP evita la fragmentación segmentando al MSS negociado.

La dirección de origen es privada, 192.168.0.29; los paquetes que llegan a Google llevan la dirección pública del router. El NAT la reescribe a la salida, que es otra cosa que la captura solo puede mostrar desde este lado.

3.4 Capa de transporte — TCP

Handshake de tres vías, tramas 280–282:

```
280 192.168.0.29 → 34.143.73.2 [SYN] seq=0 win=64240 MSS=1460
281 34.143.73.2 → 192.168.0.29 [SYN,ACK] seq=0 ack=1 win=65535 MSS=1412
282 192.168.0.29 → 34.143.73.2 [ACK] seq=1 ack=1 win=259
```

Segmentación. Cada lado anuncia el segmento más grande que está dispuesto a recibir. Nosotros ofrecemos 1,460 — la MTU Ethernet estándar de 1,500 menos 20 bytes de cabecera IP y 20 de TCP. Google responde 1,412, más pequeño porque su red agrega encapsulación propia. Gana el menor de los dos, así que los segmentos mayores de la captura llevan exactamente 1,412 bytes de carga.

Por eso una respuesta de `tools/list` de 3,680 bytes no puede viajar en un paquete: a 1,412 por segmento necesita tres, y TCP los reensambla antes de que la capa HTTP vea un solo mensaje. JSON-RPC no se entera de nada de esto.

Reutilización de conexión. Esta sesión abrió **11 conexiones TCP distintas** para 19 mensajes, y cada petición llevó `Connection: close`. Eso es una propiedad del cliente, no de MCP: `tools/conformance_check.py` está construido sobre `urllib`, que no agrupa conexiones. El anfitrión del proyecto usa `httpx`, que mantiene una viva durante toda la sesión — así que la misma conversación lanzada desde `python -m host.main` cuesta un

handshake en lugar de once. Para un protocolo tan conversador como MCP, esa es la diferencia entre pagar un viaje de ida y vuelta por llamada y pagarlo una sola vez.

3.5 Capa de aplicación — HTTP, JSON-RPC, y lo que TLS oculta

De captures/mcp_local_plaintext.pcapng, un intercambio completo reensamblado:

```
POST /mcp HTTP/1.1
Content-Type: application/json
Accept: application/json, text/event-stream
Mcp-Session-Id: d4e3271d8a84425aaee04e287afb091c
Mcp-Protocol-Version: 2025-11-25
Connection: close

{"jsonrpc": "2.0", "id": 3, "method": "tools/list"}
```

```
HTTP/1.1 200 OK
server: uvicorn
content-type: application/json
content-length: 3680

{"jsonrpc": "2.0", "id": 3, "result": {"tools": [...]}}
```

En la captura contra Cloud Run, en cambio:

```
Trama 283 TLS handshake tipo 1 (Client Hello)
server_name: netops-mcp-261683462697.us-central1.run.app
Trama 288 TLS handshake tipo 2 (Server Hello)
```

Después del Server Hello todo es Application Data. La extensión SNI del Client Hello viaja antes de que empiece el cifrado, así que un observador aprende **a qué host** se le está hablando, pero ni un byte de lo que se le dice. Ni el método HTTP, ni una cabecera, ni un mensaje JSON-RPC.

Conviene enunciarlo como resultado y no como inconveniente: **desplegar el servidor detrás de TLS volvió el protocolo inobservable para todo el mundo, nosotros incluidos.**

3.6 Clasificación de los mensajes (requisito 7)

De la captura en claro se recuperaron 19 mensajes JSON-RPC, cada uno en una de las tres categorías que pide el enunciado:

Categoría	Cantidad	Cuáles
Sincronización	4	initialize, notifications/initialized, 2× ping
Petición	6	tools/list, 4× tools/call, resources/list
Respuesta	9	5 exitosas, 4 de error

Tres códigos reservados distintos aparecen en esa única sesión: -32600, -32601 y -32602.

La notificación se distingue sin leer una sola línea de JSON. En la captura, `notifications/initialized` es respondida con `202 Accepted` y `content-length: 0`, mientras cada request del mismo archivo recibe `200` con cuerpo. La diferencia entre los dos tipos de mensaje es observable en el cable.

El análisis detallado, trama por trama, está en `captures/analysis.md`.

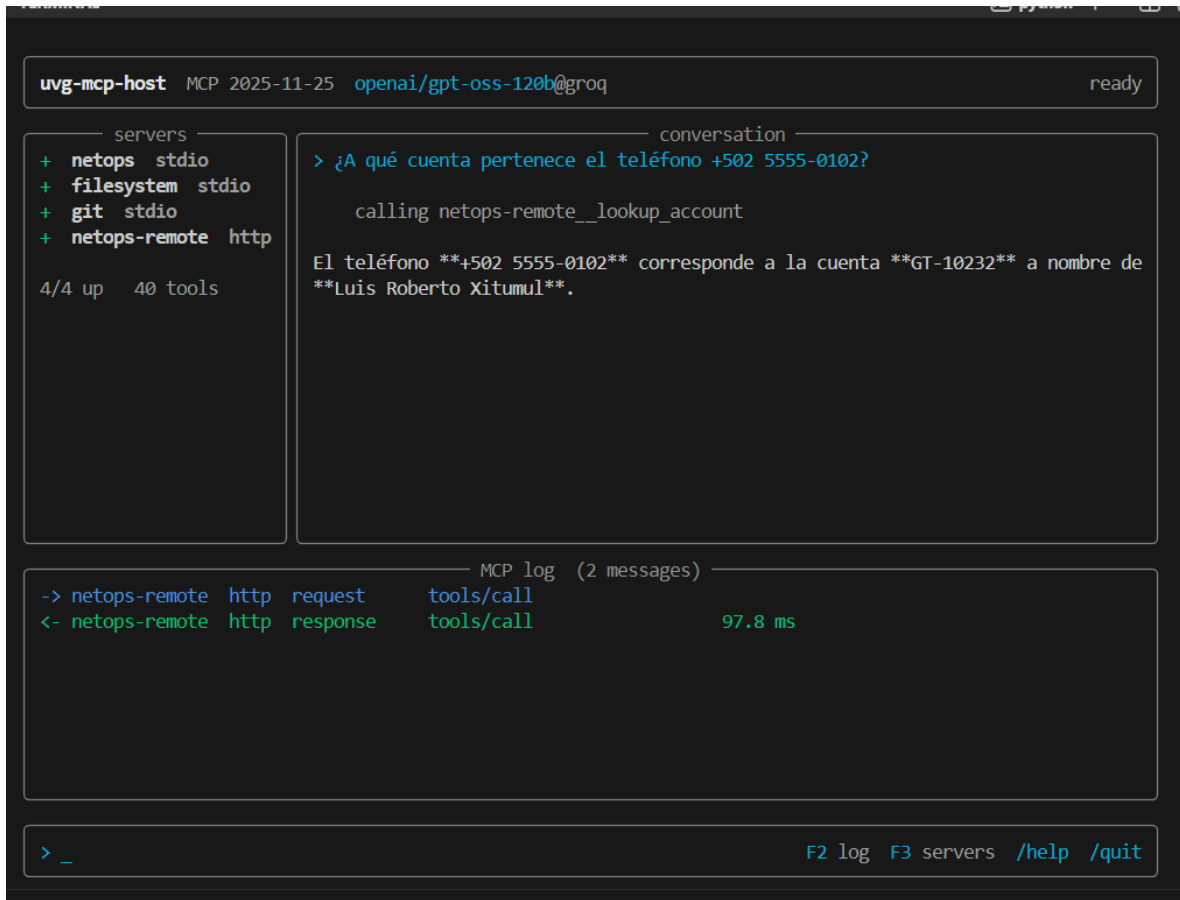
3.7 Cómo se anidan las capas

JSON-RPC 2.0	<code>{"jsonrpc":"2.0","id":3,"method":...}</code>	el mensaje
MCP	<code>Mcp-Session-Id</code> , <code>Mcp-Protocol-Version</code>	la semántica
HTTP/1.1	<code>POST /mcp</code> , <code>200 / 202</code> , <code>Content-Type</code>	el portador
TLS	(solo remoto) cifra todo lo de arriba	
TCP	puertos, <code>seq/ack</code> , <code>MSS 1412</code> , segmentación	
IPv4	<code>192.168.0.29 → 34.143.73.2</code> , <code>TTL</code> , <code>DF</code>	
Ethernet II	<code>MAC del host → MAC del gateway</code> , <code>MTU 1500</code>	

JSON-RPC no define un transporte. La especificación dice cómo se ve un request, una response y una notification, y absolutamente nada sobre cómo viajan. MCP es la capa que elige un portador, y ofrece dos: stdio con framing NDJSON, y HTTP. Los mensajes son byte a byte los mismos en cualquiera de los dos — la respuesta de `tools/list` de esta captura es el mismo JSON que el servidor stdio escribe a su tubería.

Por eso la pregunta del framing pertenece al transporte y no al mensaje. Sobre stdio, donde una tubería es apenas un flujo de bytes, MCP necesita una regla para saber dónde termina un mensaje, y elige un objeto JSON por línea. Sobre HTTP la pregunta nunca surge: `Content-Length` ya dice cuánto mide el cuerpo, así que a MCP no le queda nada que decidir.

4. La interfaz y su justificación de HCI (punto extra)



4.1 Feedback inmediato · visibilidad del estado del sistema

La primera heurística de Nielsen es que el sistema mantenga informado al usuario de lo que está pasando. Un bucle agéntico es justo el caso que lo necesita: el modelo puede pasar veinte segundos llamando cuatro herramientas, y sin una línea de estado el usuario no puede distinguir eso de un cuelgue.

El indicador nombra, no solo gira. Un spinner dice “algo está pasando”. La cabecera dice * netops-remote__run_diagnostic [2/10]: qué herramienta, en qué servidor, y cuánto queda del presupuesto de diez iteraciones. Esa última cifra también previene la ansiedad de no saber si el sistema se detendrá alguna vez.

La decisión técnica que lo hace posible: **el layout nunca se baja**. Un Live de rich y un input() bloqueante no pueden compartir la terminal, y el arreglo habitual es destruir la pantalla para preguntar y reconstruirla después — lo que hace desaparecer la interfaz exactamente cuando el usuario está decidiendo qué hacer. Aquí las teclas se leen de a una y se dibujan en el pie como parte del mismo repintado que todo lo demás, así que preguntar no es otro modo: es otro frame.

4.2 Psicología del color · y por qué el color nunca va solo

La paleta es semántica y tiene exactamente tres categorías:

Color	Significado
Azul	cliente → servidor
Verde	servidor → cliente
Rojo	error

El rojo **gana** sobre la dirección. Cuando uno lee una traza, el fallo es lo que está buscando, así que pesa más que la pregunta de quién envió el mensaje.

Tres categorías y no siete: una paleta que distingue siete cosas no distingue ninguna. Tres es lo que un lector puede sostener mientras escanea, y mapea exactamente sobre las tres preguntas que el log responde — ¿lo enviamos?, ¿respondieron?, ¿funcionó?

Pero el color nunca es el único portador. Toda representación lleva la misma información en una forma que sobrevive perderlo:

- -> y <- cargan la dirección
- +, ! y - cargan el estado de cada servidor
- el tipo de mensaje va escrito en su propia columna

Alrededor del 8% de los hombres tiene una deficiencia en la visión de rojo y verde — precisamente el par sobre el que esta paleta más se apoya. Un reporte impreso es monocromo. Un proyector de salón lava los colores saturados. En los tres casos la traza tiene que seguir siendo legible, y lo es, porque el color es redundante aquí: hace la estructura más rápida de ver, nunca posible de ver.

Esa redundancia **está impuesta por pruebas, no por buenas intenciones**. El panel lateral se renderiza en las pruebas con `color_system=None` y se exige que los glifos estén presentes; se exige además que cada estado tenga un glifo distinto. El marco de la interfaz usa grises precisamente para no competir: si los bordes fueran azules, el azul dejaría de significar algo.

4.3 Agrupación · principios de Gestalt

Región común. Qué servidores están arriba y cuántas herramientas aportan son una sola pregunta, así que viven en una sola caja con su propio borde. La alternativa — cuarenta nombres de herramienta por el costado — sería técnicamente más información y prácticamente menos, porque nadie sostiene cuarenta nombres. El panel resume: 4/4 up 40 tools.

Proximidad. Dentro del transcript, una llamada a herramienta va sangrada y atenuada bajo la respuesta a la que pertenece, porque es algo que el asistente hizo *camino a* una respuesta, no una respuesta.

Diseño minimalista · detalle bajo demanda. La traza es el artefacto más interesante que produce este proyecto y también la forma más rápida de volver la pantalla ilegible. Por eso está a una tecla de distancia (F2) en lugar de siempre encendida o siempre apagada. Al colapsarla, las filas se las queda la conversación.

4.4 Prevención de errores

La quinta heurística prefiere que un problema no ocurra a que se reporte bien.

- `/call` **parsea su argumento JSON antes de que nada llegue al cable**, así que una coma de más vuelve como un mensaje legible en vez de un `-32700` desde el servidor.
- `/log tail abc` responde “no es un número” en lugar de lanzar una excepción.
- `ctrl+c` con texto escrito **limpia la línea**; solo sale si la línea ya está vacía. La acción destructiva se reserva para cuando el usuario ya confirmó que no hay nada que perder.
- Toda salida de herramienta se escapa antes de dibujarse: la descripción de un ticket que contenga corchetes se muestra literal y no puede reestilizar —ni borrar— la pantalla.

4.5 Reconocimiento antes que recuerdo

Los atajos y el estado de los cuatro servidores están en pantalla de forma permanente. Nadie debería tener que recordar que `/log tail 20` existe. En el CLI, `/servers` imprime una tabla que se pierde en el scroll; en la TUI el estado **está ahí mientras escribís**, que es una propiedad de interfaz, no de formato.

4.6 Dos errores que solo aparecieron al mirar

Vale la pena registrarlos porque ninguno se habría encontrado razonando sobre el código.

La tabla convertida en confeti. La consola de captura estaba fija en 100 columnas mientras el panel que la recibe mide `terminal - 26 - 4`. En una terminal de 110 eso son 80: cada línea sobraba veinte columnas, se envolvía dentro del panel, y dejaba de ser una tabla. Ahora el ancho se calcula de la consola viva y se recalcula antes de cada comando.

La conversación que desaparecía. Un panel dibuja su contenido desde arriba y pierde lo que no cabe. El panel de log ya cortaba su cola; la conversación no. Una respuesta larga llenaba el panel y **todo turno posterior se dibujaba fuera de la vista**: el anfitrión funcionaba perfectamente y el usuario no podía ver nada de eso. Ahora el

transcript se recorre hacia atrás hasta agotar el presupuesto de filas, de modo que el turno más nuevo siempre está en pantalla y el más viejo es el que cede.

Ambos tienen prueba de regresión.

5. Cumplimiento de los requisitos

#	Requisito	Evidencia
1	Chatbot responde preguntas generales vía LLM	sesión de demostración
2	Mantiene contexto de la sesión	«¿Quién fue Alan Turing?» → «¿En qué fecha nació?» → <i>23 de junio de 1912</i>
3	Log completo de interacciones MCP, visible	logs/*.jsonl + /log tail, panel en vivo
4	Filesystem y Git conectados, con escenario	docs/demo-filesystem-git.md
5	Servidor propio local con especificación	servers/netops/SPEC.md
6	El mismo servidor en la nube, consumido igual	19/19 contra Cloud Run
7	Captura con mensajes clasificados	captures/analysis.md §3.6
8	Reporte: especificación y endpoints	§2 de este documento
9	Reporte: análisis por capas	§3 de este documento
10	Reporte: conclusiones	§6
—	README en inglés	README.md
—	Commits graduales	34 commits, 19 ago – 17 sep
—	Cero SDK de MCP	test automatizado sobre requirements.txt
—	UI con justificación HCI	§4 de este documento

5.1 La prueba de que local y remoto son el mismo servidor

El requisito 6 pide el mismo servidor desplegado remotamente y consumido igual. La forma de demostrarlo no es señalar el código sino correr la misma verificación independiente contra los tres entornos:

```
python tools/conformance_check.py → 19/19
python tools/conformance_check.py --http http://localhost:8080/mcp → 19/19
python tools/conformance_check.py --http https://...run.app/mcp → 19/19
```

`conformance_check.py` es deliberadamente un cliente **ajeno**: usa solo la biblioteca estándar y no importa nada de `host/`. Si compartiera nuestro codificador, un bug en ese codificador se cancelaría solo y la verificación pasaría igual. La mitad HTTP está construida sobre `urllib` y no sobre `httpx` por la misma razón: la librería por la que habla nuestro anfitrión no puede ser la que dé fe del servidor.

6. Conclusiones y comentarios (requisito 10)

6.1 Lo que el proyecto enseñó sobre el protocolo

La separación entre error de protocolo y error de dominio es lo que hace funcionar al agente. Es fácil verla como una formalidad de la especificación hasta que uno observa al modelo recuperarse solo. En la demostración de Filesystem y Git, el modelo intentó agregar al índice un archivo que todavía no existía; el servidor respondió con un resultado exitoso que cargaba `isError`, el modelo lo leyó y escribió el archivo primero. Después le habló al servidor de Filesystem con la ruta del servidor de Git, fue rechazado, llamó a `list_allowed_directories` para averiguar cuál era la raíz, y corrigió. Ninguno de los dos fallos llegó al usuario como un error. Si hubieran sido errores de protocolo, el intercambio nunca habría estado bien formado y no habría habido nada que leer.

JSON-RPC es deliberadamente incompleto, y eso es una virtud. No dice nada sobre transporte, y por eso el mismo mensaje viaja por una tubería y por HTTPS sin cambiar un byte. La incomodidad aparece donde la especificación calla: sobre stdio hubo que elegir un framing, y sobre HTTP hubo que inventar sesiones — porque una tubería es la sesión y una petición HTTP no.

El transporte es una decisión con costo medible, no un detalle. En una sesión con los cuatro servidores:

filesystem	stdio	initialize	9195.7 ms	npx resolviendo su paquete
git	stdio	initialize	6435.0 ms	uvx resolviendo el suyo
netops-remote	http	initialize	2727.9 ms	arranque en frío de Cloud Run
netops-remote	http	tools/call	98.3 ms	régimen, por Internet
netops	stdio	tools/call	0.6 ms	régimen, por tubería

Las cuatro primeras cifras salen de `docs/cloud-run-session.jsonl`; la última, de `docs/mcp-session-sample.jsonl`.

El arranque en frío del servidor **en la nube es más rápido que levantar el servidor local de npx**. stdio paga arrancar un proceso y, para `npx` y `uvx`, resolver un paquete; HTTP paga un viaje a un contenedor que la plataforma levanta. Ya en caliente, la relación se invierte: 98 ms contra 1 ms. Ese es el intercambio honesto, y la razón de que el transporte sea una elección.

6.2 Lo que salió mal, y qué se aprendió

El timeout que confundía dos operaciones distintas. El host usaba un único presupuesto de 30 segundos para conectarse y para llamar. Cuando `mcp-server-git` publicó la versión 1.30.0 entre dos sesiones, `uvx` tuvo que descargarla en frío, el handshake pasó de 30 segundos y el host descartó un servidor que funcionaba perfectamente — reportando “*no response to initialize*”, que apunta al servidor y no a nuestro propio presupuesto. Conectarse y llamar no son la misma clase de operación: un servidor `npx` puede tener que **descargarse a sí mismo**, mientras que una llamada a un proceso ya corriendo jamás debería tardar treinta segundos. Ahora tienen presupuestos separados.

Una caída que habría arruinado la presentación. El modelo contestó «300 Mbps» con un espacio fino `U+202F`; la consola de Windows codifica en `cp1252`, no puede representarlo, y `rich` lanzaba `UnicodeEncodeError` **a media respuesta**, matando el CLI. El transporte stdio ya forzaba UTF-8 en los procesos hijos; el anfitrión no lo hacía con sus propias salidas. Es la clase de fallo que solo aparece según lo que conteste el modelo, o sea la peor forma de fallar en vivo.

Medir mal es peor que no medir. Al investigar la lentitud de `uvx` se obtuvieron tiempos de 41 y 55 segundos usando `Start-Job` de PowerShell, que levanta un `runspace` entero y contamina la medición. Medido correctamente, el servidor tardaba 2.6 segundos. La conclusión provisional era falsa y habría llevado a arreglar el problema equivocado.

6.3 Limitaciones conocidas

Vale más enunciarlas que dejar que se descubran.

El estado no es durable en la nube. Los tickets se escriben en `/tmp`, que es por instancia y desaparece cuando Cloud Run la recicla. Es un límite deliberado: el enunciado pide el mismo servidor alcanzable remotamente, no una base de datos gestionada. Un despliegue durable apuntaría `NETOPS_DATA_DIR` a un volumen montado o reemplazaría el almacén JSON por uno respaldado en base de datos — y `core.py` no cambiaría en ninguno de los dos casos.

La captura en texto plano es sobre loopback. Por las razones de §3.1, los mensajes JSON-RPC legibles y las tramas Ethernet reales vienen de capturas distintas.

El servidor HTTP no implementa el modo SSE. La especificación permite responder con un flujo de eventos y ofrecer `GET /mcp` para mensajes iniciados por el servidor.

Nada en este proyecto necesita que un servidor empuje: todo intercambio es una petición del cliente y su respuesta.

El tier gratuito de Groq limita agresivamente. Las demostraciones con el modelo tardan minutos de reloj esperando reintentos. El anfitrión aguanta el límite y reintenta solo, pero para una presentación en vivo conviene usar la API de Anthropic, que se cambia con una línea en `.env`.

6.4 Comentario final

Lo que más costó de este proyecto no fue ninguno de los algoritmos. Fue la disciplina de no confundir capas: que el transporte no supiera de MCP, que MCP no supiera del modelo, que el modelo no supiera si un servidor está en esta máquina o en Iowa. Cada vez que esa disciplina se relajó, el síntoma apareció en otro lado — el prompt del sistema que mencionaba un ISP y desorientaba al modelo cuando las herramientas venían de Git; el timeout único que descartaba un servidor sano; el ancho de consola fijo que destrozaba una tabla.

La prueba de que la separación quedó bien puesta no es que el código se vea ordenado. Es que `registry.py` es el único archivo del anfitrión que sabe que un servidor puede ser remoto, y que agregar el transporte HTTP no obligó a cambiar ni una línea de `MCPClient`, del bucle agéntico, ni de la interfaz.

Referencias

- JSON-RPC 2.0 — <https://www.jsonrpc.org/specification>
- MCP Specification 2025-11-25 — <https://modelcontextprotocol.io/specification/2025-11-25>
- Servidores oficiales de MCP — <https://github.com/modelcontextprotocol/servers>
- Anthropic Messages API — <https://docs.claude.com/en/api/overview>
- Nielsen, J. — *10 Usability Heuristics for User Interface Design*, Nielsen Norman Group
- Cloud Run — <https://cloud.google.com/run/docs>

Uso de IA generativa. Este proyecto se desarrolló con asistencia de Claude (Anthropic) para implementación, revisión y redacción, bajo el reglamento de UVG. Todas las decisiones de diseño, las verificaciones y las mediciones reportadas aquí fueron ejecutadas y comprobadas sobre el código real del repositorio.